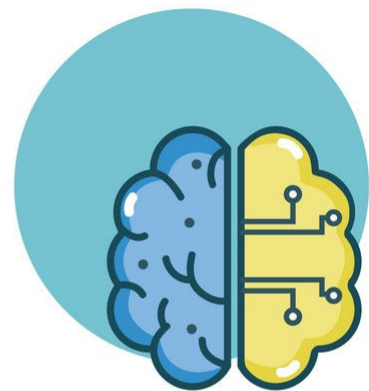


INTRODUCTION TO MACHINE LEARNING

CLUSTERING



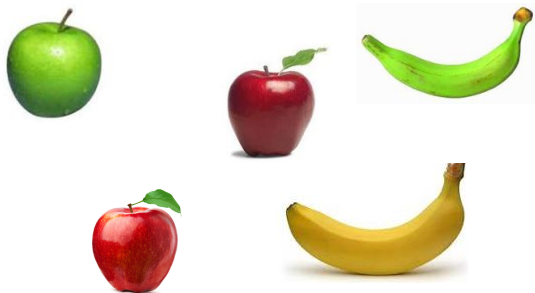
Elisa Ricci



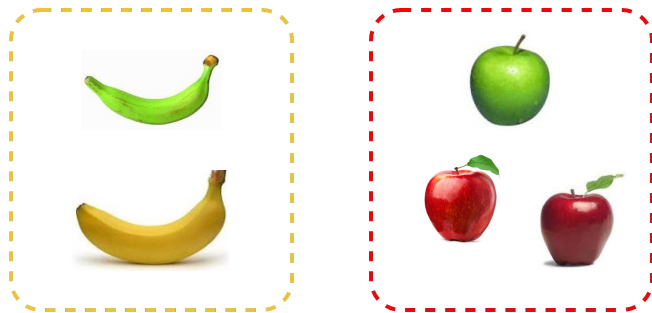
UNSUPERVISED LEARNING: CLUSTERING

Goal: Discover hidden structures in data without pre-defined labels.

UNLABELED INPUT



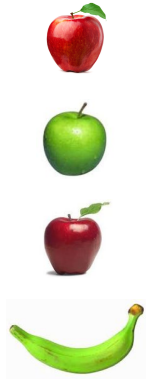
CLUSTERING OUTPUT



In **Unsupervised Learning**, we are given data (examples) but **no labels**.

UNSUPERVISED LEARNING: CLUSTERING

Raw Data



Feature
Extraction

Feature Vectors

$f_1, f_2, \dots, f_n$

$f_1, f_2, \dots, f_n$

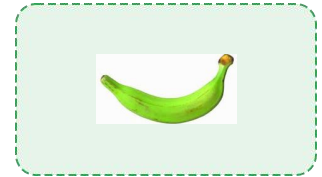
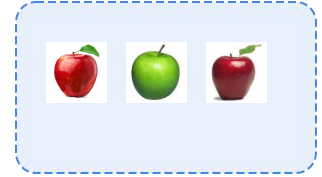
$f_1, f_2, \dots, f_n$

$f_1, f_2, \dots, f_n$



Clustering
Algorithm

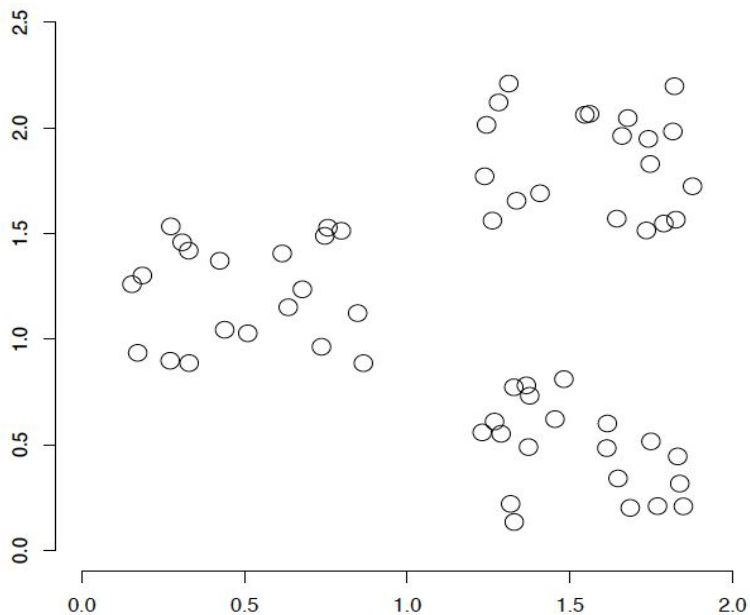
Clusters



In **Unsupervised Learning**, we are given data but **no labels** (no supervision).

UNSUPERVISED LEARNING: CLUSTERING

Clustering: the process of grouping a set of objects into classes of similar objects

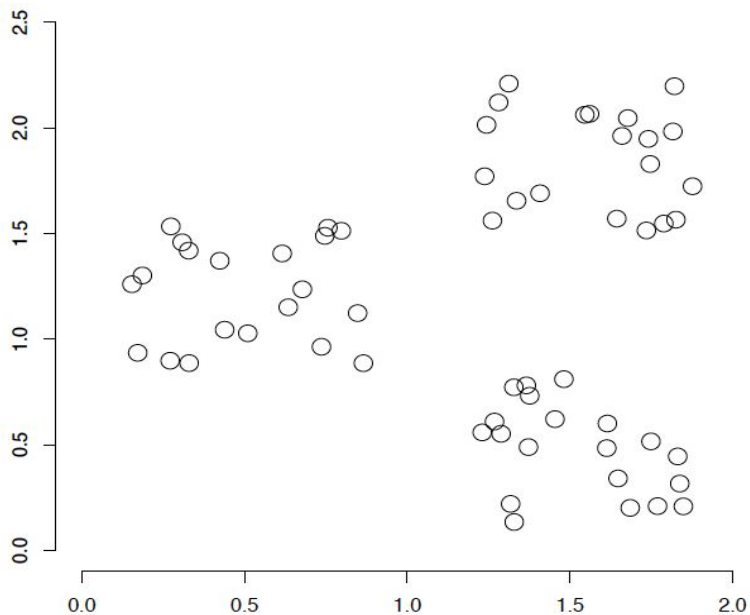


Which clustering algorithms can we use?

What are the issues for clustering?

UNSUPERVISED LEARNING: CLUSTERING

Clustering: the process of grouping a set of objects into classes of similar objects



Which clustering algorithms can we use?

K-MEANS CLUSTERING

The most popular and widely used clustering algorithm in unsupervised learning.



1. Initialization

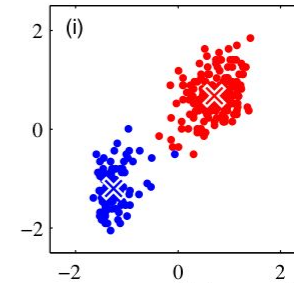
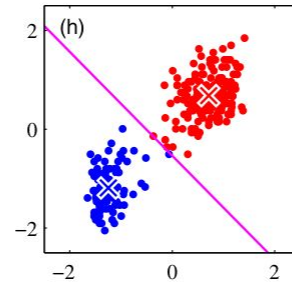
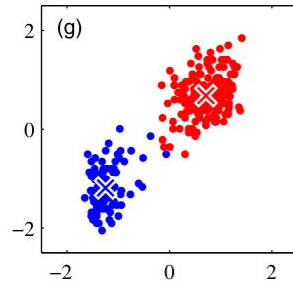
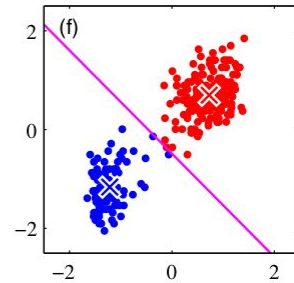
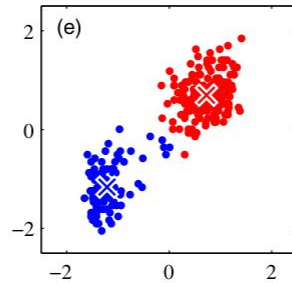
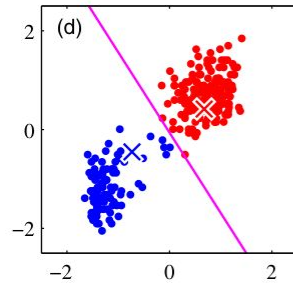
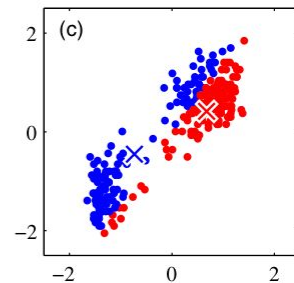
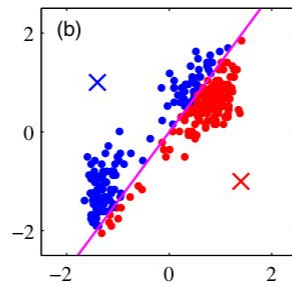
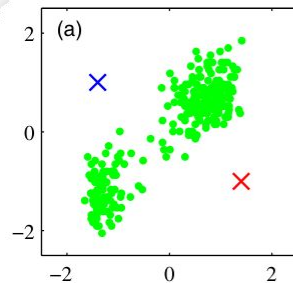
Start with some initial cluster centroids chosen randomly or via heuristics.



2. Iterative Optimization

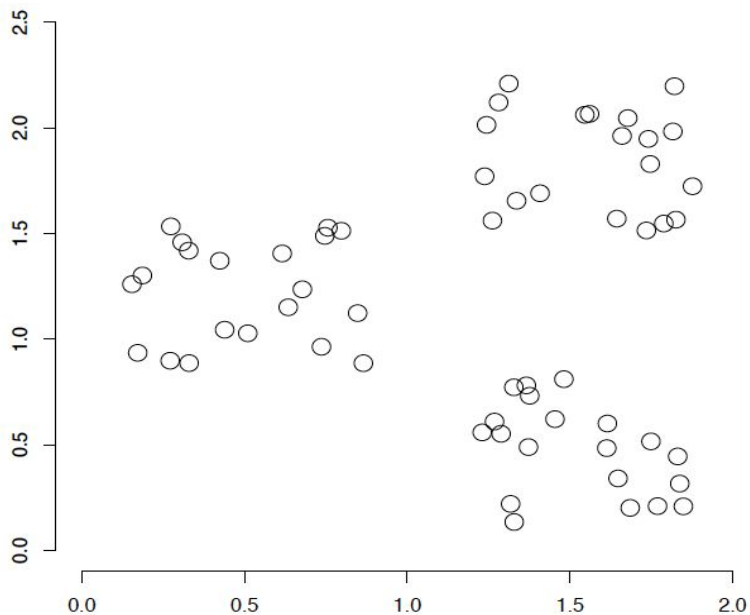
- **Assignment:** Assign each example to its closest centroid.
- **Update:** Recalculate centers as the mean of all points in the cluster.

K-MEANS



UNSUPERVISED LEARNING: CLUSTERING

Clustering: the process of grouping a set of objects into classes of similar objects



What are the issues
for clustering?

ISSUES FOR CLUSTERING

Representation

How do we represent examples? Typically expressed as feature vectors.

Similarity Metric

Which distance or similarity measure should be used between examples?

Clustering Type

Should the structure be flat (partitions) or hierarchical (trees)?

Number of Clusters

Is 'K' fixed a priori by the user or is it discovered as data-driven?

CLUSTERING ALGORITHMS

Flat Algorithms

- K-means clustering
- Spectral clustering



Hierarchical Algorithms

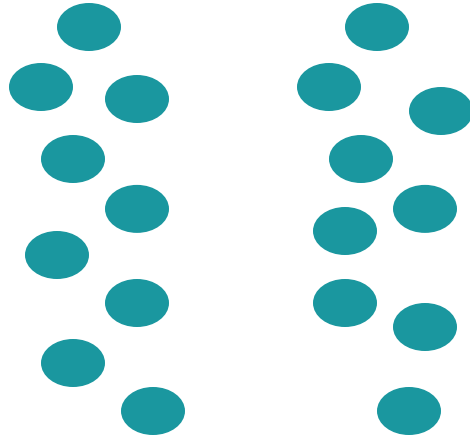
- Bottom-up, agglomerative
- Top-down, divisive



HARD VS. SOFT CLUSTERING

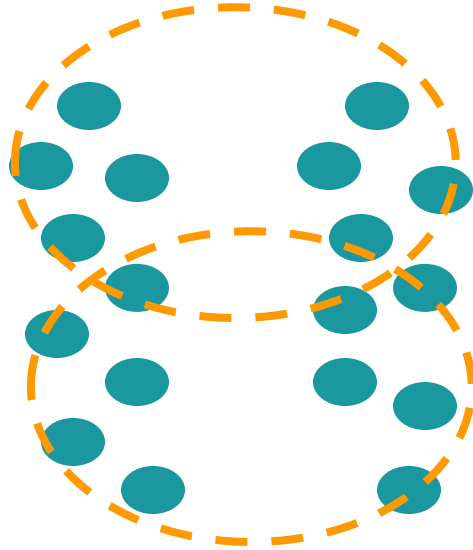
- **Hard clustering:** Each example belongs to exactly one cluster
 - K-Means
- **Soft clustering:** An example can belong to more than one cluster (probabilistic)
 - Makes more sense for applications like creating browsable hierarchies, e.g. you may want to put a pair of sneakers in two clusters: (i) sports apparel and (ii) shoes

PROBLEMS WITH K-MEANS



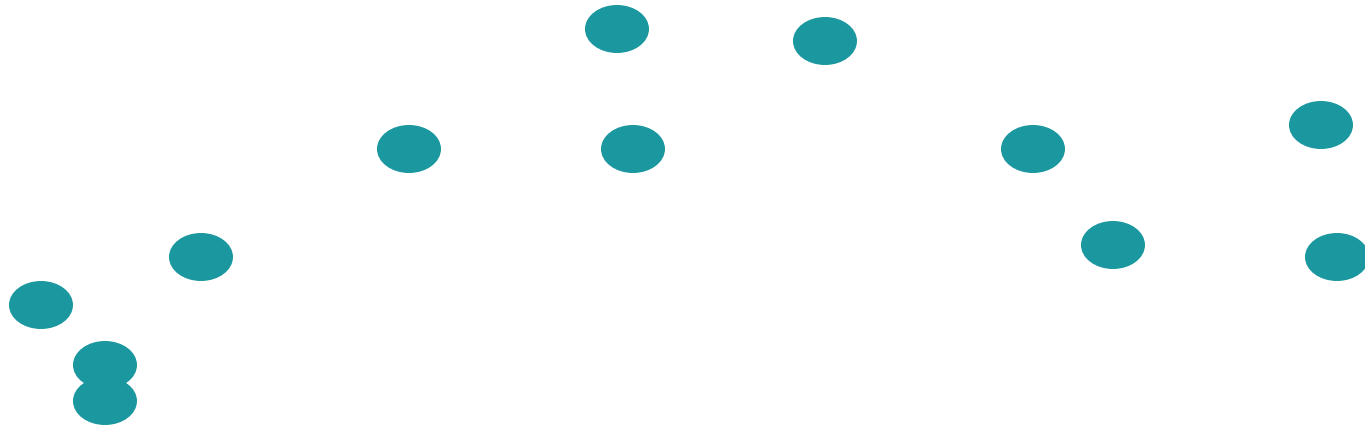
What would K-means give us here?

PROBLEMS WITH K-MEANS

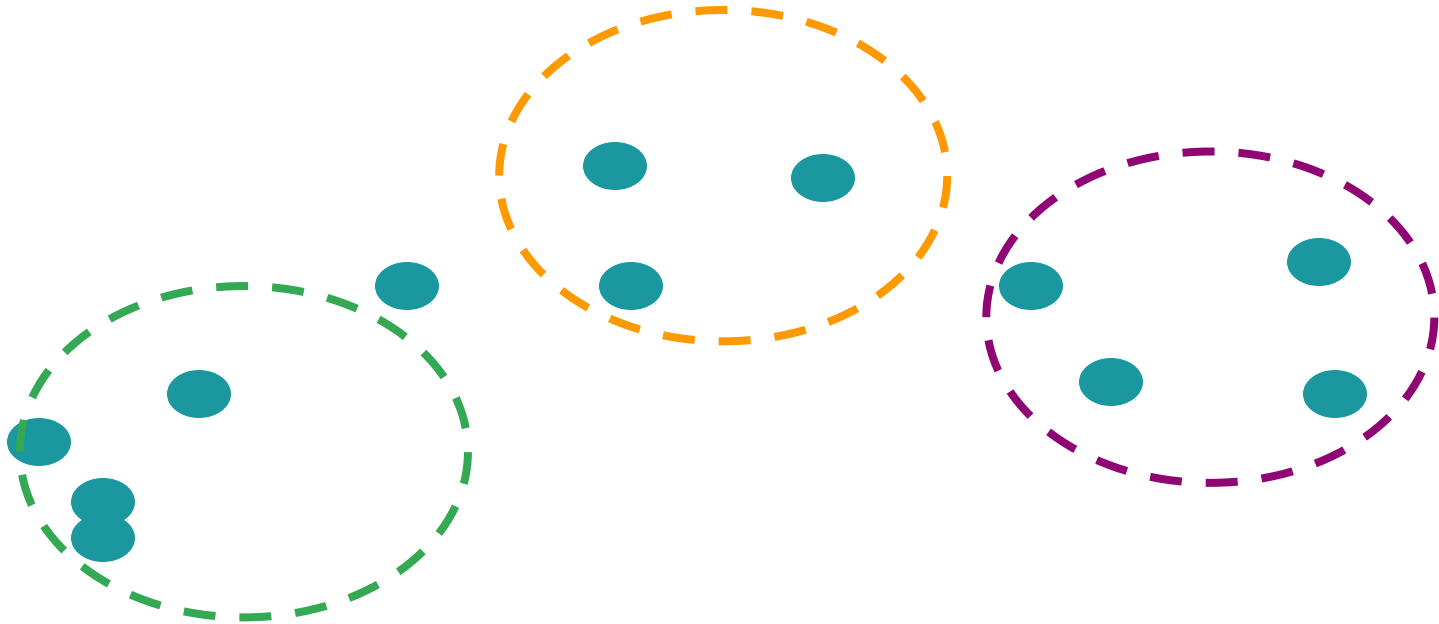


k-means assumes **spherical clusters!**

SPHERICAL CLUSTERS



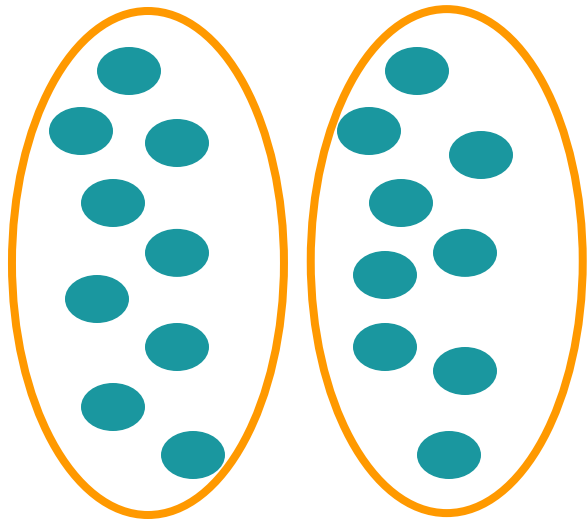
SPHERICAL CLUSTERS



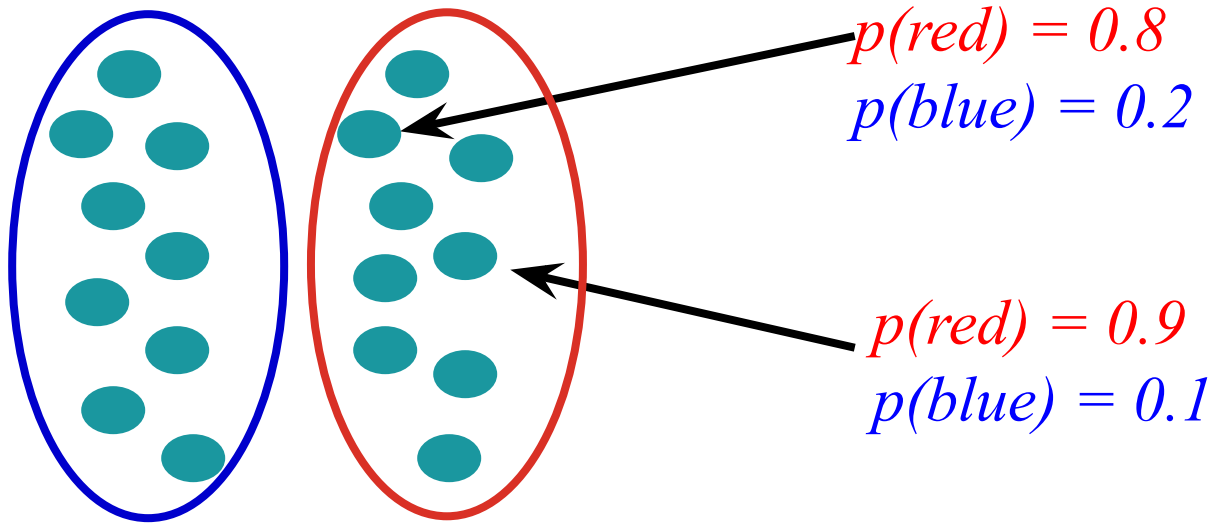
k-means assumes **spherical clusters**!

A BETTER ALGORITHM: EM CLUSTERING

Expectation Maximization (EM): assume data came from a **mixture of Gaussians** (elliptical data), assign data to cluster with a certain probability (**soft clustering**)



SOFT CLUSTERING



EM CLUSTERING

- Very similar at a high-level to K-means
 - Iterate between assigning points and recalculating cluster centers
- Two main differences between K-means and EM clustering
 - We assume **elliptical clusters** (instead of spherical)
 - It is a **soft clustering** algorithm

EM CLUSTERING PROCESS

Initialization

Start with some initial cluster centers

E-Step

Expectation / Soft Assignment

Soft assign points to each cluster:

Calculate the probability of each point belonging to each cluster

M-Step

Maximization / Update

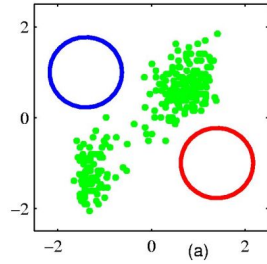
Recalculate the cluster centers:

Calculate new cluster parameters:

Maximum likelihood cluster centers given the current soft clustering

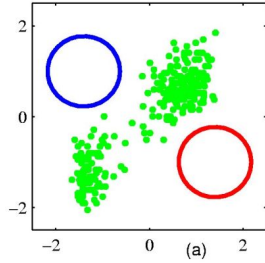
Iterate until convergence between E and M steps

EXAMPLE



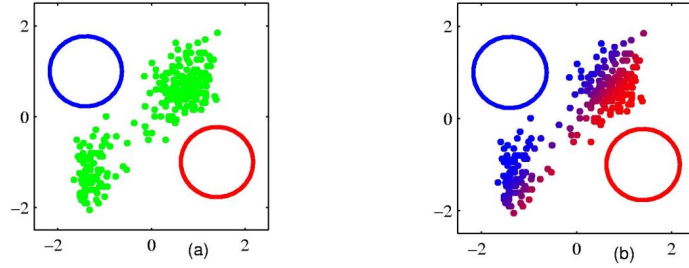
Start with some initial cluster centers

EXAMPLE



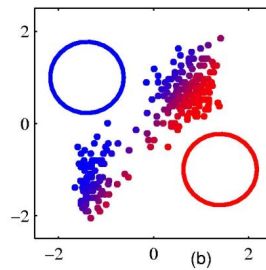
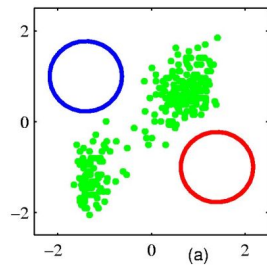
Which points belong to which clusters (soft)?

EXAMPLE



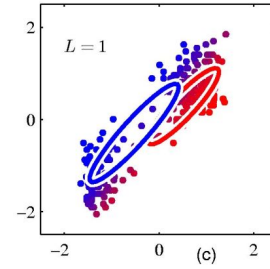
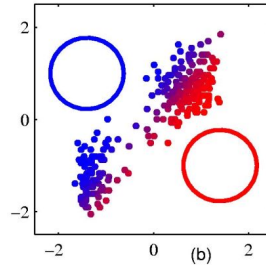
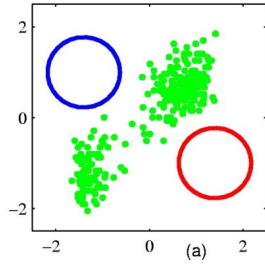
It is a soft (probabilistic) assignment

EXAMPLE



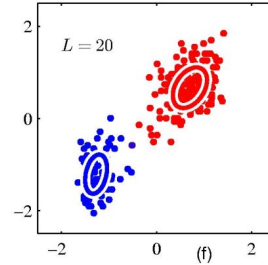
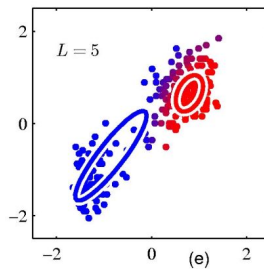
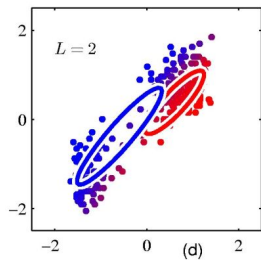
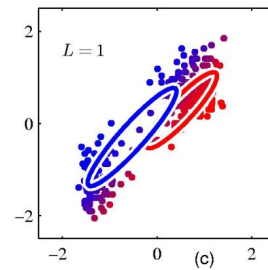
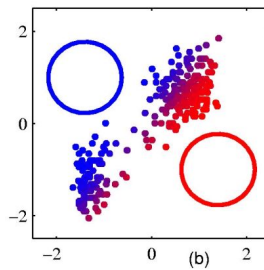
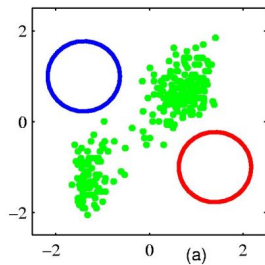
What do the new centers look like?

EXAMPLE



Cluster centers get a **weighted** contribution from points

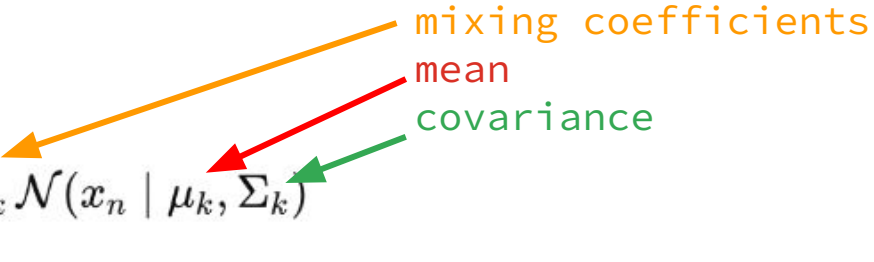
EXAMPLE



... after several iterations

MODEL

The model is parameterized as:

$$p(x_n) = \sum_{k=1}^K \pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k)$$


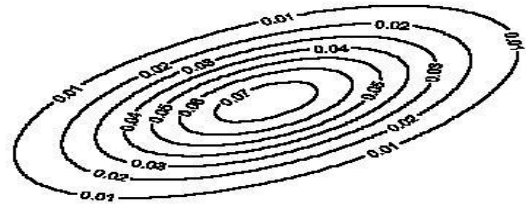
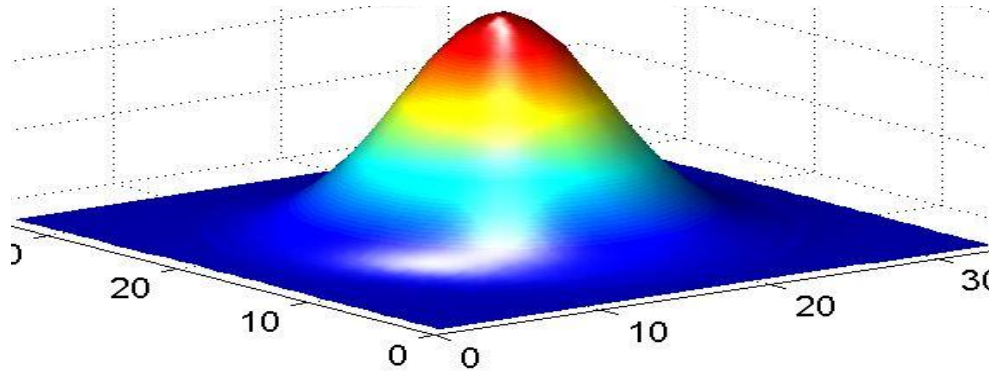
A diagram with three colored arrows pointing to the equation above. An orange arrow points from the text 'mixing coefficients' to the π_k term. A red arrow points from the text 'mean' to the μ_k term. A green arrow points from the text 'covariance' to the Σ_k term.

mixing coefficients
mean
covariance

with constraints:

$$\sum_{k=1}^K \pi_k = 1, \quad \pi_k \geq 0$$

RECAP: A GAUSSIAN IN 2D



Covariance determines
the shape of these contours

EM CLUSTERING PROCESS

Initialization

Start with some initial cluster centers

E-Step

Expectation / Soft Assignment

Soft assign points to each cluster:

Calculate the probability of each point belonging to each cluster

M-Step

Maximization / Update

Recalculate the cluster centers:

Calculate new cluster parameters:

Maximum likelihood cluster centers given the current soft clustering

Iterate until convergence between E and M steps

INITIALIZATION & E-STEP

Initialize parameters

$$\{\pi_k^{(0)}, \mu_k^{(0)}, \Sigma_k^{(0)}\}_{k=1}^K$$

E-STEP

Compute responsibilities

for each data point

and component k :

$$\gamma_{nk} = P(z_n = k \mid x_n) = \frac{\pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_n \mid \mu_j, \Sigma_j)}$$

M-STEP

Compute effective number of points
assigned to each component:

$$N_k = \sum_{n=1}^N \gamma_{nk}$$

Update mixing coefficients

$$\pi_k = \frac{N_k}{N}$$

Update means

$$\mu_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} x_n$$

Update covariances

$$\Sigma_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (x_n - \mu_k)(x_n - \mu_k)^T$$

EXPECTATION MAXIMIZATION: SUMMARY

- Similar to K-Means:
 - E-step is the 'assignment' step; M-step is the 'update' step
- Each iterations increases the likelihood of the data and is guaranteed to converge (though to a local optimum)
- The number of components K is still a free parameter

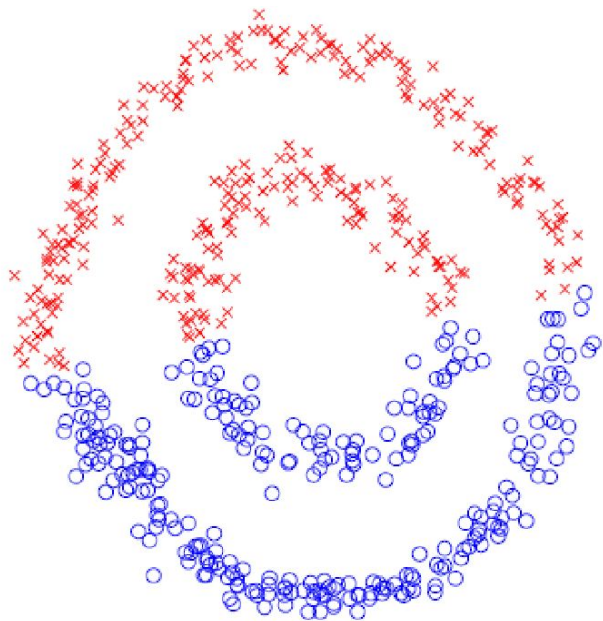
EXPECTATION MAXIMIZATION: SUMMARY

- Multiple restarts are recommended; the result can differ substantially between runs.
- Not just for clustering while K-means is just for clustering
- EM is a general purpose approach for training a model when you do not have labels

OTHER CLUSTERING ALGORITHMS

- K-means and EM-clustering are by far the most popular for clustering
- However, they cannot handle all clustering tasks
- What types of clustering problems cannot they handle?

NON-GAUSSIAN DATA: THE CHALLENGE



The Problem

K-means and EM struggle with complex, non-spherical geometries.

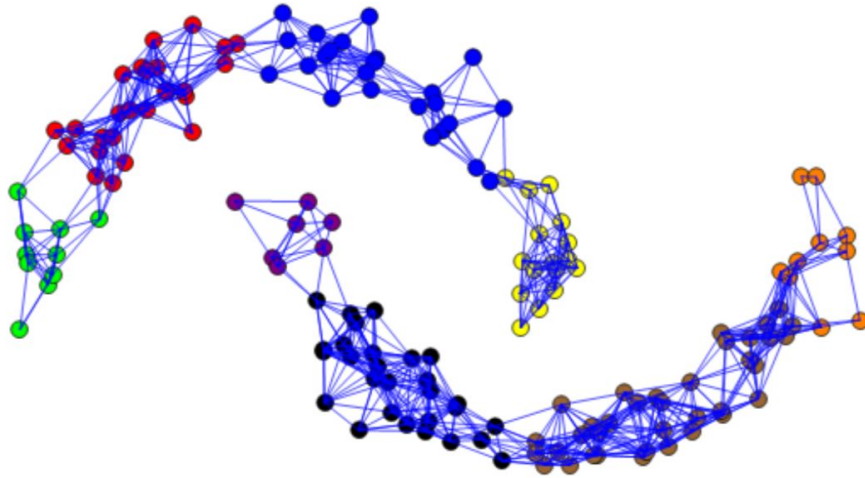
Decision Logic

Like classification: **Global** (Linear) vs. **Local** (K-NN) boundaries.

Solution: Spectral Clustering

SPECTRAL CLUSTERING

Group points based on links in a graph



STEP 1: CREATE A GRAPH

- We can create a **fully connected graph** or a **K-nearest neighbor graph** (each node is only connected to its K closest neighbors)
- We can create a **graph with some notion of similarity among nodes**. It is common to use a Gaussian Kernel

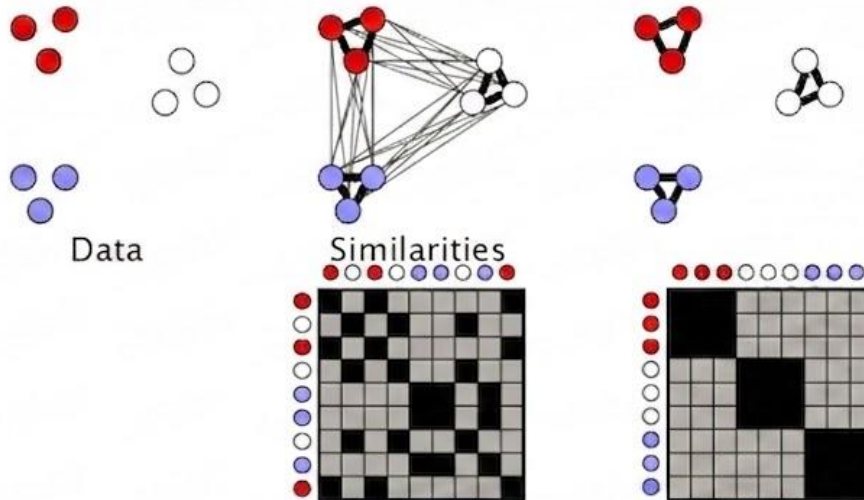
$$W(i, j) = \exp\left(\frac{-|x_i - x_j|^2}{\sigma^2}\right)$$



$O(n^2)$ edges — expensive for large datasets

STEP 2: GRAPH PARTITIONING

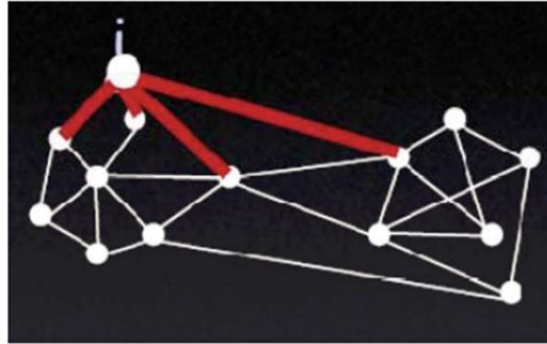
Define a **similarity matrix** and **cut the graph** (partition the graph into K groups by removing the fewest or lightest edges):



GRAPH TERMINOLOGY

Degree of a node

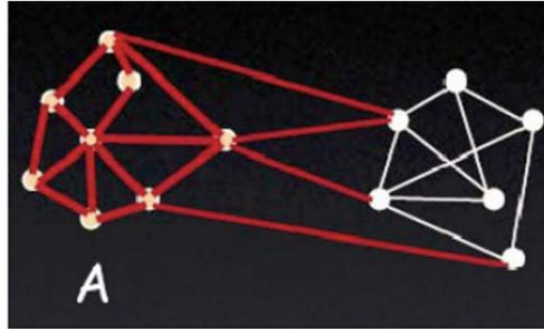
$$d_i = \sum_j w_{i,j}$$



GRAPH TERMINOLOGY

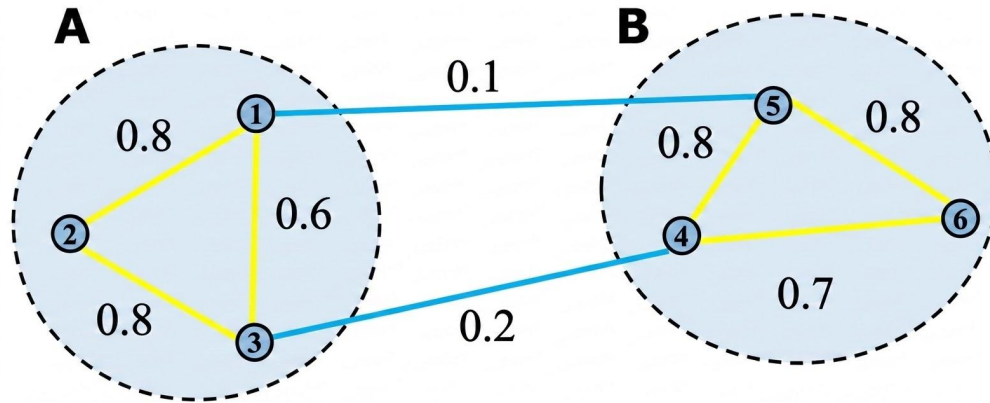
Volume of a set

$$\text{vol}(A) = \sum_{i \in A} d_i, A \subseteq V$$



GRAPH CUT

- Consider a partition of the graph into two parts A and B
- $Cut(A, B)$: sum of the weights of the set of edges that connect the two groups
- An intuitive goal is **find the partition that minimizes the cut**



$$Cut(A, B) = 0.3$$

NORMALIZED CUT

- Minimize

$$NCut(A, B) = Cut(A, B)/Vol(A) + Cut(A, B)/Vol(B)$$

- Normalise by cluster volume: discourages tiny, isolated partitions.
- This problem can formalized as a discrete optimization problem and then relaxed in the continuous domain and become a generalized eigenvalue problem

SPECTRAL CLUSTERING

Normalized Cut is the theoretical foundation and the first algorithm, the Ng et al. algorithm is a practical variant.

[Link](#)

On Spectral Clustering: Analysis and an algorithm

Andrew Y. Ng
CS Division
U.C. Berkeley
ang@cs.berkeley.edu

Michael I. Jordan
CS Div. & Dept. of Stat.
U.C. Berkeley
jordan@cs.berkeley.edu

Yair Weiss
School of CS & Engr.
The Hebrew Univ.
yuweiss@cs.huji.ac.il

Abstract

Despite many empirical successes of *spectral clustering* methods—algorithms that cluster points using eigenvectors of matrices derived from the data—there are several unresolved issues. First, there are a wide variety of algorithms that use the eigenvectors in slightly different ways. Second, many of these algorithms have no proof that they will actually compute a reasonable clustering. In this paper, we present a simple spectral clustering algorithm that can be implemented using a few lines of Matlab. Using tools from matrix perturbation theory, we analyze the algorithm, and give conditions under which it can be expected to do well. We also show surprisingly good experimental results on a number of challenging clustering problems.

1 Introduction

The task of finding good clusters has been the focus of considerable research in machine learning and pattern recognition. For clustering points in $\mathbb{R}^n$ —a main application focus of this paper—one standard approach is based on generative models, in which algorithms such as EM are used to learn a mixture density. These approaches suffer from several drawbacks. First, to use parametric density estimators, harsh simplifying assumptions usually need to be made (e.g., that the density of each cluster is Gaussian). Second, the log likelihood can have many local minima and therefore multiple restarts are required to find a good solution using iterative algorithms. Algorithms such as K-means have similar problems.

A promising alternative that has recently emerged in a number of fields is to use *spectral methods* for clustering. Here, one uses the top eigenvectors of a matrix derived from the distance between points. Such algorithms have been successfully used in many applications including computer vision and VLSI design [5, 1]. But despite their empirical successes, different authors still disagree on exactly which eigenvectors to use and how to derive clusters from them (see [11] for a review). Also, the analysis of these algorithms, which we briefly review below, has tended to focus on simplified algorithms that only use one eigenvector at a time.

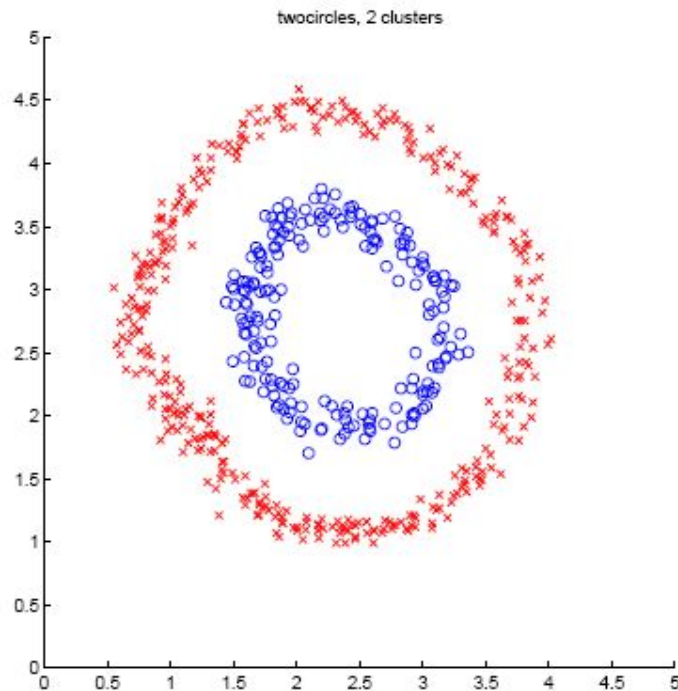
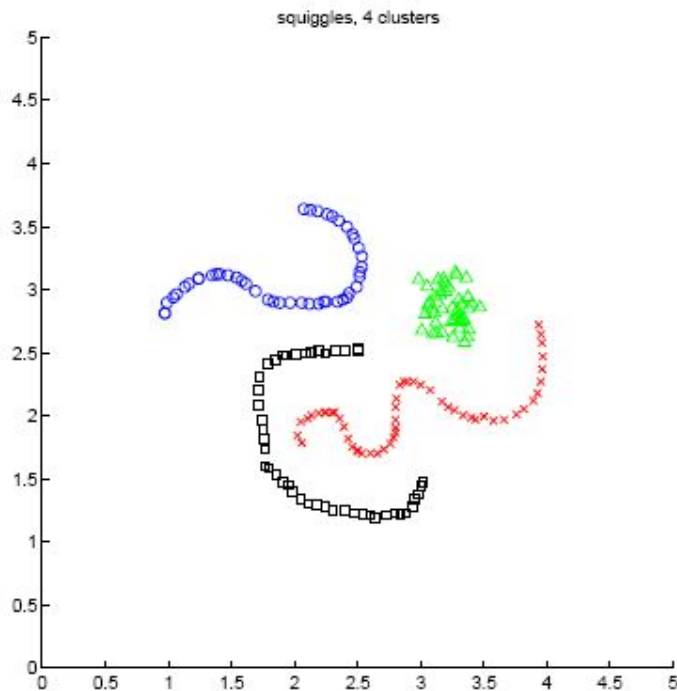
One line of analysis makes the link to *spectral graph partitioning*, in which the sec-

SPECTRAL CLUSTERING

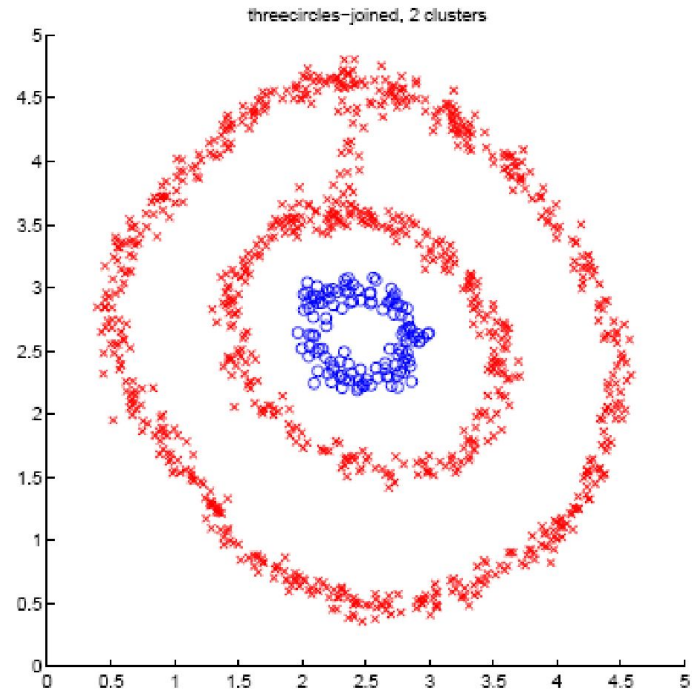
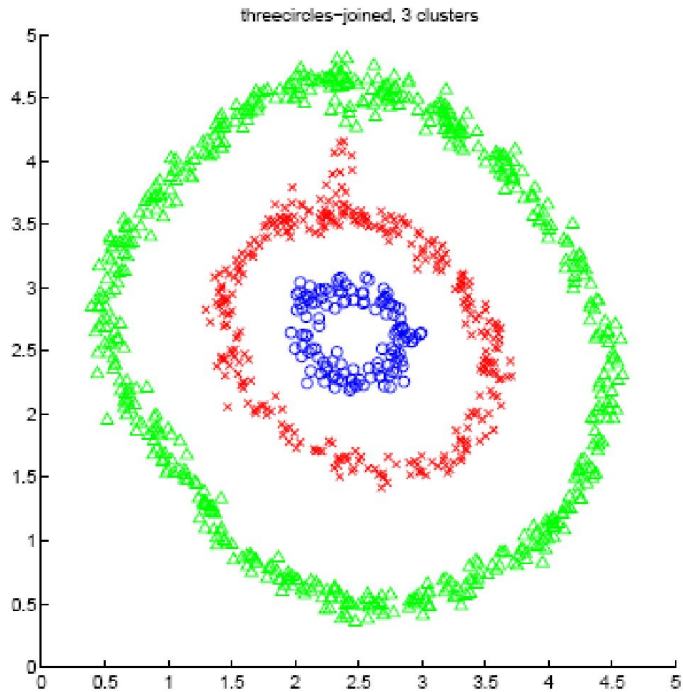
Given a set of points $S = \{s_1, \dots, s_n\}$ in $\mathbb{R}^d$ that we want to cluster into k subsets:

1. Form the affinity matrix $A \in \mathbb{R}^{n \times n}$ defined by $A_{ij} = \exp(-\|s_i - s_j\|^2 / 2\sigma^2)$ if $i \neq j$, and $A_{ii} = 0$.
2. Define D to be the diagonal matrix whose (i, i) -element is the sum of A 's i -th row, and construct the matrix $L = D^{-1/2} A D^{-1/2}$.
3. Find $x_1, x_2, \dots, x_k$, the k largest eigenvectors of L (chosen to be orthogonal to each other in the case of repeated eigenvalues), and form the matrix $X = [x_1 x_2 \dots x_k] \in \mathbb{R}^{n \times k}$ by stacking the eigenvectors in columns.
4. Form the matrix Y from X by renormalizing each of X 's rows to have unit length (i.e. $Y_{ij} = X_{ij} / (\sum_j X_{ij}^2)^{1/2}$).
5. Treating each row of Y as a point in $\mathbb{R}^k$, cluster them into k clusters via K-means or any other algorithm (that attempts to minimize distortion).
6. Finally, assign the original point s_i to cluster j if and only if row i of the matrix Y was assigned to cluster j .

SPECTRAL CLUSTERING

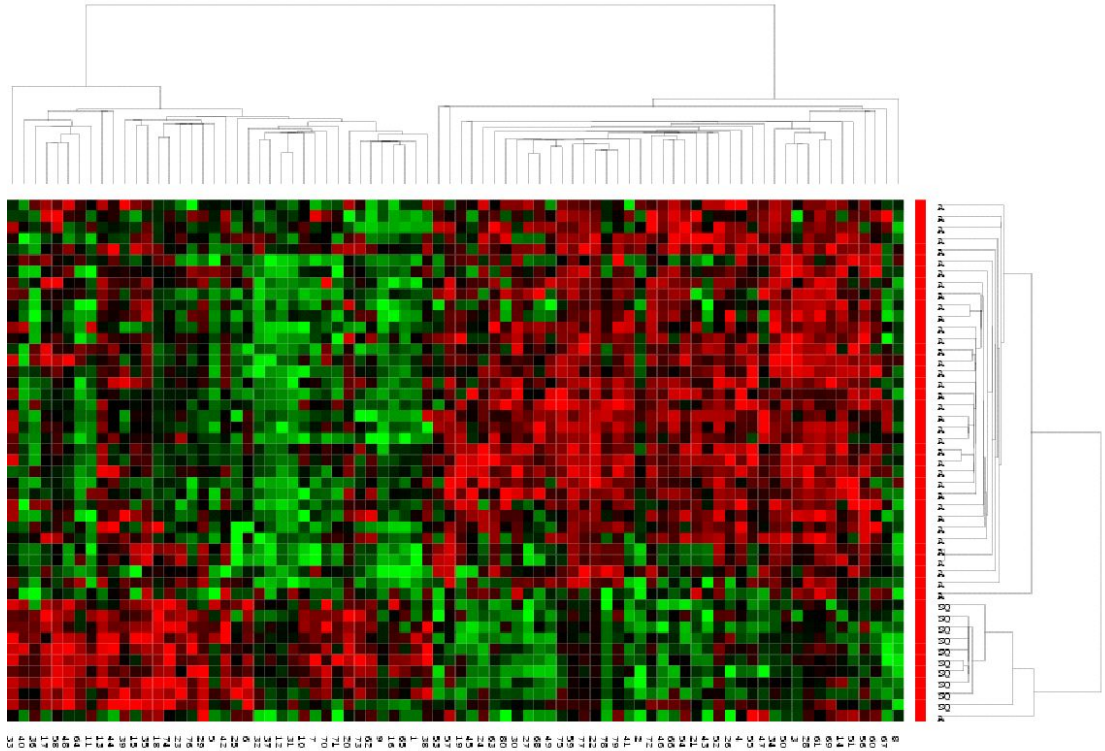


SPECTRAL CLUSTERING



HIERARCHICAL CLUSTERING

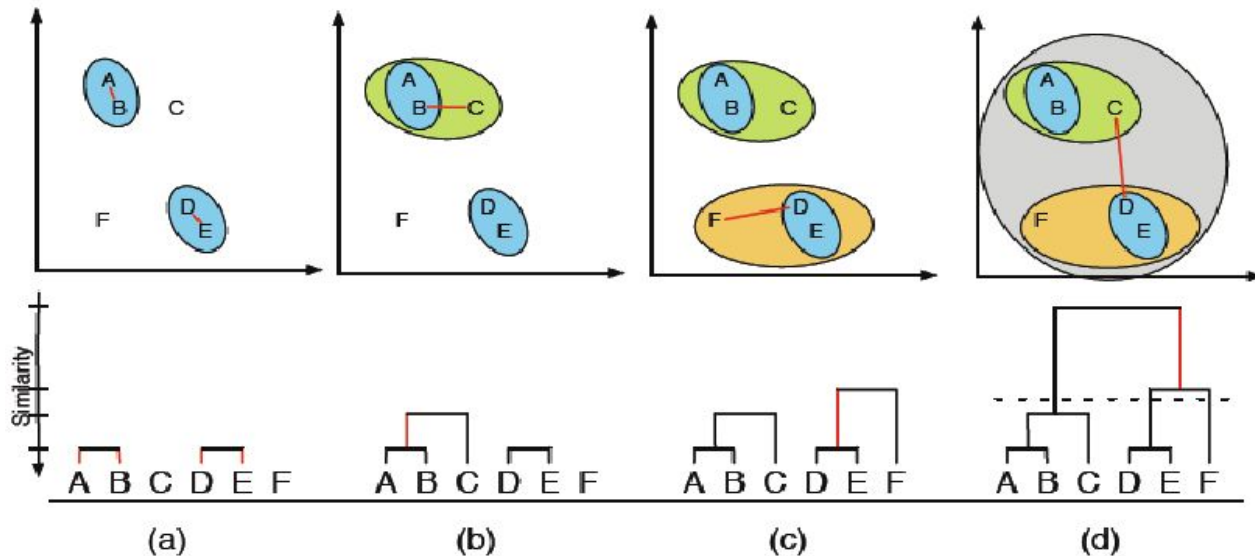
Gene profiles



HIERARCHICAL CLUSTERING

Produces a set of nested clusters organized as a hierarchical tree

The tree is called **dendrogram**

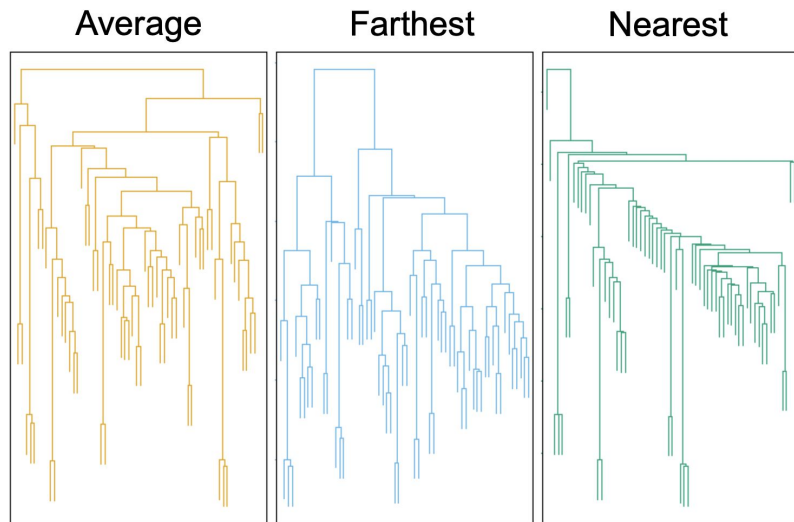
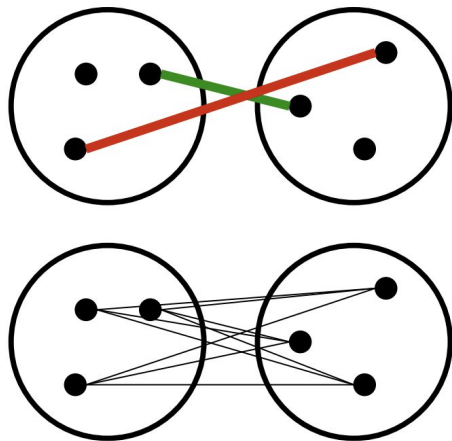


AGGLOMERATIVE CLUSTERING

- **Idea:** first merge very similar instances and then incrementally build larger clusters out of smaller clusters
- **Algorithm:**
 - Initially, each instance in its own cluster
 - Repeat:
 - Pick the two closest clusters
 - Merge them into a new cluster
 - Stop when there is only one cluster left
- Produces not one clustering, but **a family of clusterings** represented by a **dendrogram**

AGGLOMERATIVE CLUSTERING

- How should we define “closest” for clusters with multiple elements?
- Different choices (closest pair, farthest pair, average among all distances, etc.) creates different solutions



CLUSTERING ALGORITHMS SUMMARY

Property	K-Means	EM	Spectral	Hierarchical (Agglom.)
Assignment	Hard	Soft (probabilistic)	Hard (via K-Means)	Hard
Cluster shape	Spherical only	Elliptical	Any shape	Any (linkage-dependent)
Requires k	Yes	Yes	Yes	No
Scalability	Excellent	Moderate	Poor	Poor
Key strength	Fast, scalable	Soft uncertain membership	Non-convex clusters	No k needed, hierarchy
Typical use	Large-scale data	Overlapping clusters	Non-linear shapes	Gene expr.

SUMMARY

Unsupervised Learning

No labels find structure. Key choices: representation, distance, flat/hierarchical, k.

K-Means

Minimise within-cluster variance. Fast but limited to spherical, hard clusters. Run multiple times.

EM Clustering

Soft, probabilistic assignments. Models elliptical clusters. Generalises K-Means. Guaranteed to maximize the log-likelihood.

Spectral Clustering

Graph-based. Handles non-convex shapes. Solves generalised eigenvalue problem on graph Laplacian.

Hierarchical

Produces a dendrogram. Agglomerative bottom-up. Linkage criterion determines cluster shape.

When to use which?

K-Means=fast; EM=soft; Spectral=shape; Hierarchical=k unknown.

Try K-Means first.

QUESTIONS?

